TaskDoor

API & Architecture Documentation

Hyperlocal Apartment Delivery Platform

Version	1.0
Date	March 2026
Stack	PHP 8 · MySQL · Tailwind CSS · Vanilla JS
Server	XAMPP · Apache mod_rewrite
Base URL	http://localhost/aprtment/
API Base	http://localhost/aprtment/api/v1/

Table of Contents

1. Platform Overview 3
2. Architecture & File Structure 4
3. Database Schema 6
4. Authentication & Authorization 8
5. API Endpoints — Authentication 9
6. API Endpoints — Orders 10
7. API Endpoints — Bids 12
8. API Endpoints — Chat 13
9. API Endpoints — Notifications 14
10. API Endpoints — Users & Profile 15
11. API Endpoints — Delivery Status 16
12. API Endpoints — Ratings 17
13. API Endpoints — Credits 18
14. API Endpoints — Location 20
15. API Endpoints — Buildings 21
16. API Endpoints — Settings 22
17. API Endpoints — Admin 23
18. Web Router (index.php) 28
19. Frontend JavaScript 30
20. Security & Error Handling 31
21. Seed Data & Setup 32

1. Platform Overview

TaskDoor is a hyperlocal apartment delivery platform that connects **customers** living in residential buildings with nearby **vendors** (shops) and **delivery agents**. Customers place orders for items; vendors compete with price bids during a timed bidding window; the customer confirms a bid; an admin assigns a delivery agent; the agent picks up and delivers the item to the apartment unit.

1.1 Order Lifecycle

Step	Status	Who Acts	Description
1	bidding	Customer	Customer places an order — vendors have 60 s to bid
2	bidding → awaiting	System	Bid window expires; system auto-selects lowest bid
3	awaiting_confirmation	Customer	Customer confirms the winning bid
4	confirmed	Admin	Admin assigns a delivery agent (costs agent 1 credit)
5	assigned	Agent	Agent accepts & picks up item from vendor
6	picked_up	Agent	Agent updates status to on_the_way
7	on_the_way	Agent	Agent updates status to delivered
8	delivered	Customer	Customer can now rate the agent (1–5 stars)
—	cancelled	Customer	Any order in bidding phase may be cancelled

1.2 User Roles

Role	Can Do	Key Fields
customer	Place orders, confirm bids, rate agents	building_id, block, floor, house_no
vendor	Submit price bids, view won orders	shop_name, shop_location, latitude, longitude
delivery_agent	Accept assignments, update delivery status, purchase credits	credits balance, agent_buildings
admin	Manage all users, buildings, packages, credit adjustments, settings	

2. Architecture & File Structure

TaskDoor is a classic **PHP MVC** application (without a full framework) served over XAMPP. A single **front controller** (index.php) routes all web page requests using PHP's match() expression. RESTful API calls go directly to PHP files under api/v1/, bypassing the front controller entirely.

2.1 Request Flow

Browser / SPA	→	Apache + .htaccess	→	index.php (router)	→	views/
Mobile / API client	→	Apache (no rewrite)	→	api/v1/*.php directly	→	JSON

2.2 Directory Layout

`/`	Root: index.php (router), .env, .htaccess, setup.php
`/api/response.php`	apiSuccess() / apiError() JSON helpers
`/api/v1/middleware/`	auth.php (JWT), cors.php, rate_limit.php
`/api/v1/auth/`	login, register, logout endpoints
`/api/v1/orders/`	create, index, show, confirm, cancel
`/api/v1/bids/`	submit
`/api/v1/chat/`	send, messages (delta polling)
`/api/v1/notifications/`	index, mark_read
`/api/v1/users/`	profile, update
`/api/v1/delivery/`	update (status transitions)
`/api/v1/ratings/`	create, runner_stats
`/api/v1/credits/`	balance, packages, purchase
`/api/v1/location/`	update-position, nearby-buildings, nearby-vendors
`/api/v1/buildings/`	index (public)
`/api/v1/settings/`	index (GET public / POST admin)
`/api/v1/admin/`	users, buildings, tasks, agents, assign, credit-packages,
	adjust-credits, building-units, credit-transactions, stats
`/app/config/`	db.php (PDO singleton), constants.php
`/app/helpers/`	jwt.php (HS256), validation.php
`/app/models/`	User, Order, Bid, Chat, Notification, Rating, Building
`/app/services/`	LocationService.php (Haversine)
`/views/`	auth/, customer/, vendor/, delivery/, admin/, shared/, layouts/
`/assets/js/`	app.js, chat.js, swipe.js, task-feed.js
`/database/`	schema.sql, seeds.sql, migration_credits.sql

3. Database Schema

Database name: **taskdoor** | Engine: MySQL InnoDB | Access: PDO singleton via getDB()

Table: buildings

Column	Type / Notes
id	INT PK AUTO_INCREMENT
name	VARCHAR(255) NOT NULL
address	TEXT
latitude / longitude	DECIMAL(10,8) / DECIMAL(11,8)
service_radius_km	DECIMAL(5,2) DEFAULT 2.00
total_floors	INT DEFAULT 10
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Table: users

Column	Type / Notes
id	INT PK AUTO_INCREMENT
name	VARCHAR(255) NOT NULL
phone	VARCHAR(20) UNIQUE NOT NULL
password_hash	VARCHAR(255) NOT NULL — BCRYPT
role	ENUM(customer|vendor|delivery_agent|admin)
building_id	FK → buildings (customers)
block / floor / house_no	VARCHAR — apartment address
shop_name / shop_location	VARCHAR — vendor details
latitude / longitude	DECIMAL — vendor/agent GPS
credits	INT DEFAULT 0 — delivery agents
is_active	TINYINT DEFAULT 1
created_at	TIMESTAMP

Table: vendor_buildings / agent_buildings

Column	Type / Notes
vendor_id / agent_id	FK → users
building_id	FK → buildings
	Composite PK — M2M assignments

Table: orders

Column	Type / Notes
id	INT PK AUTO_INCREMENT
customer_id	FK → users

building_id / block / floor / house_no	Delivery address
customer_phone	VARCHAR(20)
item_name / quantity / notes	Order details
bid_window_ends_at	TIMESTAMP — bidding deadline
selected_bid_id / selected_vendor_id	FK → bids / users
item_price / delivery_fee / total_price	DECIMAL(10,2)
delivery_agent_id	FK → users
status	ENUM(bidding|awaiting_confirmation|confirmed|assigned|picked_up|on_the_way|delivered|cancelled)
created_at / confirmed_at / assigned_at / delivered_at	TIMESTAMP lifecycle

Table: bids

Column	Type / Notes
id	INT PK AUTO_INCREMENT
order_id	FK → orders
vendor_id	FK → users
price	DECIMAL(10,2) NOT NULL
available	TINYINT DEFAULT 1
is_selected	TINYINT DEFAULT 0
submitted_at	TIMESTAMP
	UNIQUE(vendor_id, order_id)

Table: notifications

Column	Type / Notes
id	INT PK AUTO_INCREMENT
user_id	FK → users
order_id	FK → orders (nullable)
type	ENUM(new_order|bid_received|bid_selected|order_confirmed|order_assigned|status_update|new_mess
title / body	VARCHAR(255) / TEXT
is_read	TINYINT DEFAULT 0
created_at	TIMESTAMP

Table: chat_messages

Column	Type / Notes
id	INT PK AUTO_INCREMENT
order_id / task_id	FK → orders
sender_id	FK → users
message	TEXT NOT NULL
is_read	TINYINT DEFAULT 0

created_at	TIMESTAMP

Table: ratings

Column	Type / Notes
id	INT PK AUTO_INCREMENT
task_id / order_id	FK → orders
customer_id / resident_id	FK → users
runner_id / agent_id	FK → users
score	TINYINT (1–5) NOT NULL
comment	TEXT nullable
created_at	TIMESTAMP
	UNIQUE(task_id) — one rating per order

Table: credit_packages

Column	Type / Notes
id	INT PK AUTO_INCREMENT
name / description	Package display info
price_birr	DECIMAL(10,2) — cost in ETB
task_count	INT — credits awarded
sort_order	INT DEFAULT 0
is_active	TINYINT DEFAULT 1
created_at / updated_at	TIMESTAMP

Table: credit_transactions

Column	Type / Notes
id	INT PK AUTO_INCREMENT
user_id	FK → users (the agent)
type	ENUM(purchase|deduction|adjustment)
amount	INT — credits changed (signed)
balance_after	INT — snapshot
description	VARCHAR(255)
package_id	FK → credit_packages (nullable)
order_id	FK → orders (nullable)
admin_id	FK → users (nullable)
created_at	TIMESTAMP

Table: settings

Column	Type / Notes

`key`	VARCHAR PRIMARY KEY — e.g. delivery_fee, bid_window_seconds
`value`	VARCHAR(255)
`updated_at`	TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

4. Authentication & Authorization

4.1 Dual Strategy

TaskDoor uses two parallel authentication strategies: **PHP Sessions** for rendered web views and **JWT Bearer tokens** for the REST API. Both are issued at login and validated independently.

4.2 JWT Implementation

Pure-PHP HS256 HMAC implementation in app/helpers/jwt.php. No Composer dependencies.

- Algorithm: HMAC-SHA256 (HS256)
- Expiry: 24 hours from issue
- Secret: loaded from .env JWT_SECRET
- Payload: { sub (user_id), role, name, building_id, iat, exp }
- Validation: signature via hash_equals() + expiry check
- Transmission: Authorization: Bearer <token> header

4.3 Role Guards

Every API endpoint calls requireAuth([ROLE_CUSTOMER, ROLE_ADMIN]) from api/v1/middleware/auth.php. Passing an empty array allows any authenticated user. Failing the check returns 401 Unauthorized or 403 Forbidden JSON immediately.

4.4 Rate Limiting

Endpoint	Limit	Window
POST /auth/register.php	10 req	per minute per IP
POST /auth/login.php	20 req	per minute per IP
All other endpoints	60 req	per minute per IP

Rate limiting uses temp files: /tmp/td_rl_<md5(ip+script)>

5. API Endpoints — Authentication

POST	`/api/v1/auth/login.php`	*None (Rate: 20/min)*	Authenticate with phone + password. Returns JWT token and sets PHP session.

Parameters

- **phone*** — string — 10–15 digit phone number
- **password*** — string — plain text (compared via password_verify)

Response

200 OK: { "success": true, "data": { "token": "...", "user": { id, name, phone, role, building_id, ... } } }

Notes

- Sets $_SESSION['user_id'], role, name, token for web view access
- Returns 401 if phone not found or password mismatch
- Returns 429 if rate limit exceeded

POST	`/api/v1/auth/register.php`	*None (Rate: 10/min)*	Create a new user account. Role-specific fields required.

Parameters

- **name*** — string — display name
- **phone*** — string — unique 10–15 digit
- **password*** — string — minimum 6 characters
- **role*** — ENUM: customer | vendor | delivery_agent
- **building_id** — INT — required for customer
- **block / floor / house_no** — string — required for customer
- **shop_name / shop_location** — string — required for vendor
- **building_ids[]** — array of INT — optional for delivery_agent

Response

201 Created: { "success": true, "data": { "token": "...", "user": { ... } } }

Notes

- Password is hashed with PASSWORD_BCRYPT before storage
- Vendor/agent building assignments inserted into M2M tables
- Returns 409 if phone already registered

POST	`/api/v1/auth/logout.php`	*PHP Session*	Destroy the current web session.

Response

200 OK: { "success": true, "message": "Logged out" }

6. API Endpoints — Orders

POST	`/api/v1/orders/create.php`	*JWT: customer*	Create a new order. Starts the bidding window and notifies eligible vendors.

Parameters

- **item_name*** — string — what to buy
- **quantity*** — integer
- **building_id*** — INT — delivery building
- **block* / floor* / house_no*** — string — delivery apartment address
- **customer_phone*** — string — contact number for delivery
- **notes** — string — optional special instructions

Response

201 Created: { "data": { "order_id": 42, "bid_window_seconds": 60, "delivery_fee": 50.00 } }

Notes

- delivery_fee read from settings table key=delivery_fee
- bid_window_ends_at = NOW() + bid_window_seconds (default 60)
- Vendors are notified via Notification::create() for each vendor reachable from the building (LocationService::getVendorsForBuilding)

GET	`/api/v1/orders/index.php`	*JWT: any role*	List orders filtered by the caller's role. Auto-processes expired bid windows on each poll.

Parameters

- **type** — string — vendor filter: open | won (default: open)
- **status** — string — admin filter: any status value
- **building_id** — INT — admin filter

Response

200 OK: { "data": [{ order objects }] }

Notes

- customer: returns only their own orders
- vendor: open = bidding-phase orders in their area; won = orders where vendor's bid was selected
- delivery_agent: orders assigned to them
- admin: all orders with optional filters
- Calls Order::processExpiredWindows() — auto-selects lowest bid for any expired-window orders

GET	`/api/v1/orders/show.php`	*JWT: any role*	Retrieve full detail for a single order including bids (when in bidding phase).

Parameters

- **id*** — INT — order ID (query string)

Response

200 OK: { "data": { order, bids: [...] } }

Notes

- Bids array only attached if: status=bidding AND caller is admin or the order's customer
- customer: can only view own orders
- delivery_agent: can only view assigned orders
- vendor: can only view orders in bidding phase

POST	`/api/v1/orders/confirm.php`	*JWT: customer*	Customer confirms the winning bid. Moves order to awaiting_confirmation → confirmed.

Parameters

- **order_id*** — INT

Response

200 OK: { "data": { "order_id": 42, "status": "confirmed" } }

Notes

- Admin is notified with type=order_confirmed to trigger agent assignment
- Returns 400 if order is not in awaiting_confirmation status
- Returns 403 if customer does not own the order

POST	`/api/v1/orders/cancel.php`	*JWT: customer*	Cancel an order. Only allowed during the bidding phase.

Parameters

- **order_id*** — INT

Response

200 OK: { "data": { "order_id": 42, "status": "cancelled" } }

Notes

- Returns 400 if order is not in bidding status

7. API Endpoints — Bids

POST	`/api/v1/bids/submit.php`	*JWT: vendor*	Submit a price bid on an open order. Only one bid per vendor per order.

Parameters

- **order_id*** — INT
- **price*** — DECIMAL — bid price in local currency (must be > 0)
- **available** — boolean — default true; false = vendor out of stock

Response

201 Created: { "data": { "order_id": 42, "price": 250.00 } }

Notes

- Returns 400 if bid window has already expired
- UNIQUE(vendor_id, order_id) — duplicate bids rejected with 409
- Bid auto-selected by Order::selectWinningBid() when window expires — lowest price where available=1 wins

7.1 Bid Selection Logic

When the bid window expires (detected lazily on next GET /orders/index.php poll or any other order fetch), Order::processExpiredWindows() runs:

- Finds all orders WHERE status='bidding' AND bid_window_ends_at < NOW()
- For each: runs Order::selectWinningBid(order_id)
- selectWinningBid: SELECT bid with MIN(price) WHERE available=1 FOR UPDATE (race-safe)
- If a winning bid found: UPDATE order SET status='awaiting_confirmation', selected_bid_id, selected_vendor_id, item_price, total_price
- Notifies the customer: type=bid_selected
- If no bids: marks order as cancelled

8. API Endpoints — Chat

POST	`/api/v1/chat/send.php`	*JWT: any*	Send a chat message tied to an order. Access restricted to order participants.

Parameters

- **task_id*** — INT — the order ID
- **message*** — string — message body

Response

201 Created: { "data": { "message_id": 7 } }

Notes

- canChat() validates: caller must be the order's customer, assigned vendor, or delivery agent
- After send, the other party receives a notification type=new_message

GET	`/api/v1/chat/messages.php`	*JWT: any*	Fetch chat messages for an order. Supports delta polling via since_id.

Parameters

- **task_id*** — INT — order ID
- **since_id** — INT — only return messages with id > since_id (for polling)

Response

200 OK: { "data": [{ id, sender_id, message, created_at, is_read }] }

Notes

- Marks fetched messages as read automatically
- Frontend polls every 3 seconds using since_id from the last received message

9. API Endpoints — Notifications

GET	`/api/v1/notifications/index.php`	*JWT: any*	Retrieve the 30 most recent notifications for the authenticated user plus unread count.

Response

200 OK: { "data": { "notifications": [...], "unread_count": 3 } }

Notes

- Frontend polls every 15 seconds and updates the badge in the navbar

POST	`/api/v1/notifications/mark_read.php`	*JWT: any*	Mark one or all notifications as read.

Parameters

- **id** — INT — specific notification ID; omit to mark all as read

Response

200 OK: { "success": true }

9.1 Notification Types

Type	Triggered By	Recipient
new_order	Customer creates order	All eligible vendors for that building
bid_received	Vendor submits bid	Customer
bid_selected	Bid window expires, winner selected	Customer
order_confirmed	Customer confirms bid	Admin
order_assigned	Admin assigns delivery agent	Customer + Agent
status_update	Agent updates delivery status	Customer
new_message	Chat message sent	Other party in the conversation
system	Admin adjusts credits / misc	Affected user

10. API Endpoints — Users & Profile

GET	`/api/v1/users/profile.php`	*JWT: any*	Get the authenticated user's profile. Delivery agents also receive building list, stats, and recent ratings.

Response

200 OK: { "data": { user, buildings (agent), stats (agent), ratings (agent) } }

POST	`/api/v1/users/update.php`	*JWT: any*	Update profile fields for the authenticated user.

Parameters

- **name** — string — display name
- **unit_number / house_no** — string — apartment number
- **building_id** — INT — reassign to different building

Response

200 OK: { "success": true }

Notes

- Only provided fields are updated (no overwrite of unprovided fields)

11. API Endpoints — Delivery Status

POST	`/api/v1/delivery/update.php`	*JWT: delivery_agent*	Update the delivery status of an assigned order. Notifies the customer at each step.

Parameters

- **order_id*** — INT
- **status*** — ENUM: picked_up | on_the_way | delivered

Response

200 OK: { "data": { "order_id": 42, "status": "picked_up" } }

Notes

- Only the assigned agent for that order can call this endpoint
- Status must follow sequence: assigned → picked_up → on_the_way → delivered
- delivered sets delivered_at = NOW()
- Customer receives a notification type=status_update at each step

11.1 Status Transition Diagram

From Status	To Status	Actor
confirmed	assigned	Admin (via /admin/assign.php)
assigned	picked_up	Delivery Agent
picked_up	on_the_way	Delivery Agent
on_the_way	delivered	Delivery Agent
bidding	cancelled	Customer

12. API Endpoints — Ratings

POST	`/api/v1/ratings/create.php`	*JWT: customer (resident)*	Rate a delivery agent after an order is delivered. One rating per order.

Parameters

- **task_id*** — INT — the completed order ID
- **score*** — INT — 1 to 5 stars
- **comment** — string — optional review text

Response

201 Created: { "success": true }

Notes

- Validates order status = delivered
- Validates caller is the customer who placed the order
- Validates an agent was actually assigned
- UNIQUE(task_id) — second attempt returns 409 Conflict

GET	`/api/v1/ratings/runner_stats.php`	*JWT: any*	Get aggregate rating statistics and recent reviews for a delivery agent.

Parameters

- **runner_id** — INT — defaults to authenticated user if omitted

Response

200 OK: { "data": { "stats": { avg_score, count }, "reviews": [...] } }

Notes

- Returns up to 10 most recent reviews

13. API Endpoints — Credits

The **Credits System** is the economic layer for delivery agents. Agents must maintain a credit balance to receive order assignments. Each admin assignment deducts 1 credit. Agents replenish credits by purchasing packages priced in Ethiopian Birr (ETB).

GET	`/api/v1/credits/balance.php`	*JWT: delivery_agent*	Get current credit balance and the 20 most recent transactions.

Response

200 OK: { "data": { "credits": 5, "transactions": [{ id, type, amount, balance_after, description, admin_name, created_at }] } }

Notes

- Transaction types: purchase | deduction | adjustment

GET	`/api/v1/credits/packages.php`	*JWT: delivery_agent*	List active credit packages available for purchase.

Response

200 OK: { "data": [{ id, name, description, price_birr, task_count }] }

Notes

- Sorted by sort_order ASC
- Only is_active=1 packages returned

POST	`/api/v1/credits/purchase.php`	*JWT: delivery_agent*	Purchase a credit package. Credits are added atomically with a transaction log entry.

Parameters

- **package_id*** — INT — ID of the package to purchase

Response

200 OK: { "data": { "credits": 8, "added": 3 } }

Notes

- Uses SELECT ... FOR UPDATE to prevent race conditions on the user row
- Inserts into credit_transactions with type=purchase
- Returns 404 if package not found or inactive
- Note: no payment gateway integrated — purchase is currently manual/trusted

13.1 Credit Flow Diagram

Event	Credit Change	Triggered By	Transaction Type
Agent purchases package	+N credits	Agent (POST /credits/purchase)	purchase
Admin assigns agent to order	-1 credit	Admin (POST /admin/assign)	deduction
Admin manual adjustment	+/- N credits	Admin (POST /admin/adjust-credits)	adjustment

14. API Endpoints — Location

POST	`/api/v1/location/update-position.php`	*JWT: any*	Store the caller's GPS coordinates and find their nearest building.

Parameters

- **latitude*** — DECIMAL — WGS-84, range -90..90
- **longitude*** — DECIMAL — WGS-84, range -180..180

Response

200 OK: { "data": { "saved": true, "coords": { lat, lng }, "nearest_building": { id, name, distance_label } } }

Notes

- Updates users.latitude, users.longitude
- nearest_building uses Haversine formula via LocationService::getNearestBuilding()
- Returns null nearest_building if no building found within service radius

GET	`/api/v1/location/nearby-buildings.php`	*None (public)*	Find all buildings within a radius of a GPS coordinate.

Parameters

- **lat* / lng*** — DECIMAL — WGS-84 coordinates
- **radius** — DECIMAL km — max 10 km, default from building.service_radius_km

Response

200 OK: { "data": [{ building, distance_km, distance_label }] } (sorted by distance ASC)

GET	`/api/v1/location/nearby-vendors.php`	*None (public)*	Find vendors serving a specific building (by building_id) or the nearest building to a GPS point.

Parameters

- **building_id** — INT — direct building lookup
- **lat / lng / radius** — DECIMAL — alternative: find nearest building first

Response

200 OK: { "data": { "building": { ... }, "vendors": [{ user, distance_label }] } }

Notes

- Vendors are matched by: (a) GPS within service_radius_km, OR (b) explicit vendor_buildings assignment
- Used in customer 'nearby shops' view and order creation vendor notification

15. API Endpoints — Buildings

GET	`/api/v1/buildings/index.php`	*None (public)*	Return all buildings. Used to populate registration and order-creation dropdowns.

Response

200 OK: { "data": [{ id, name, address, latitude, longitude, total_floors }] }

16. API Endpoints — Settings

GET	`/api/v1/settings/index.php`	*None (public)*	Read platform-wide settings.

Response

200 OK: { "data": { "delivery_fee": "50.00", "bid_window_seconds": "60" } }

POST	/api/v1/settings/index.php	JWT: admin	Update one or more settings.

Parameters

- **delivery_fee** — DECIMAL — flat delivery charge per order
- **bid_window_seconds** — INT — how long vendors have to bid (default 60)

Response

200 OK: { "success": true }

Notes

- Uses INSERT ... ON DUPLICATE KEY UPDATE for atomic upsert

17. API Endpoints — Admin

All admin endpoints require **JWT: admin** role. Non-admin access returns 403 Forbidden.

17.1 User Management

GET	`/api/v1/admin/users.php`	*JWT: admin*	List users with optional filters. Fetch single user detail with ?id=X.

Parameters

- **id** — INT — single user detail (includes credit transactions for agents)
- **role** — ENUM — filter by role
- **search** — string — name/phone search
- **limit** — INT — default 50

Response

200 OK: { "data": { "users": [...], "role_counts": { customer:N, vendor:N, ... } } }

POST	`/api/v1/admin/users.php`	*JWT: admin*	Perform user management actions via action field.

Parameters

- **action*** — ENUM: toggle_active | update_user | reset_password | create_user | delete_user
- **user_id*** — INT — target user (except create_user)
- **(create fields)** — name, phone, password, role — for create_user action
- **(update fields)** — any updatable user column — for update_user action
- **new_password*** — string — for reset_password action

Response

200 OK: { "success": true, "data": { ... } }

Notes

- delete_user is prevented for admin accounts

17.2 Buildings Management

GET	`/api/v1/admin/buildings.php`	*JWT: admin*	List all buildings.

Response

200 OK: { "data": [{ building objects }] }

POST	/api/v1/admin/buildings.php	JWT: admin	Create a new building.

Parameters

- **name*** — string
- **address** — string
- **latitude / longitude** — DECIMAL — GPS coordinates
- **service_radius_km** — DECIMAL — default 2.00
- **total_floors** — INT — default 10

Response

201 Created: { "data": { "id": 4, ... } }

PUT	`/api/v1/admin/buildings.php`	*JWT: admin*	Update an existing building.

Parameters

- **id*** — INT
- **(any building field)** — updated value

Response

200 OK: { "success": true }

DELETE	`/api/v1/admin/buildings.php`	*JWT: admin*	Delete a building.

Parameters

- **id*** — INT

Response

200 OK: { "success": true }

17.3 Order & Agent Management

GET	`/api/v1/admin/tasks.php`	*JWT: admin*	List orders/tasks with filters.

Parameters

- **status** — ENUM — filter by order status
- **building_id** — INT
- **limit / offset** — INT — pagination

Response

200 OK: { "data": [{ order objects }] }

GET	`/api/v1/admin/agents.php`	*JWT: admin*	List delivery agents sorted by credit balance descending.

Parameters

- **building_id** — INT — filter agents serving a specific building

Response

200 OK: { "data": [{ agent + credits }] }

POST	`/api/v1/admin/assign.php`	*JWT: admin*	Assign a delivery agent to a confirmed order. Deducts 1 credit from the agent.

Parameters

- **order_id*** — INT — must be in confirmed status
- **agent_id*** — INT — must have credits >= 1

Response

200 OK: { "data": { "order_id": 42, "agent_id": 5, "agent_credits_left": 3 } }

Notes

- Uses SELECT ... FOR UPDATE on the user row to prevent race conditions
- Inserts credit_transaction with type=deduction
- Notifies both agent (order_assigned) and customer (order_assigned)
- Returns 400 if agent has 0 credits
- Returns 400 if order is not in confirmed status

17.4 Credit Administration

GET	`/api/v1/admin/credit-packages.php`	*JWT: admin*	List all credit packages (active and inactive).

Response

200 OK: { "data": [{ packages }] }

POST	`/api/v1/admin/credit-packages.php`	*JWT: admin*	Manage credit packages.

Parameters

- **action*** — ENUM: create | update | delete
- **name* / description / price_birr* / task_count*** — — for create
- **id*** — INT — for update/delete
- **(update fields)** — name, description, price_birr, task_count, is_active, sort_order

Response

200 OK: { "success": true }

POST	`/api/v1/admin/adjust-credits.php`	*JWT: admin*	Manually adjust a delivery agent's credit balance.

Parameters

- **user_id*** — INT — must be delivery_agent role
- **amount*** — INT — positive to add, negative to deduct (floor: 0)
- **description*** — string — reason for adjustment

Response

200 OK: { "data": { "new_balance": 5 } }

Notes

- Balance cannot go below 0 — deductions are capped
- Transaction logged as type=adjustment

GET	`/api/v1/admin/credit-transactions.php`	*JWT: admin*	View full credit transaction history.

Parameters

- **user_id** — INT — filter by specific agent
- **limit** — INT — default 50

Response

200 OK: { "data": [{ transaction + user_name + admin_name }] }

17.5 Building Units & Dashboard

GET	`/api/v1/admin/building-units.php`	*JWT: admin*	Get floor/block/unit breakdown for a building.

Parameters

- **building_id*** — INT

Response

200 OK: { "data": { "building": {...}, "blocks": [...], "floors": [...], "taken_units": [...], "total_floors": 10 } }

GET	`/api/v1/admin/stats.php`	*JWT: admin*	Dashboard statistics.

Notes

- user_stats: total users, count by role, inactive count
- order_stats: bidding/confirmed/delivering/delivered (today + all-time), cancelled
- recent_orders: last 20 orders
- credit_stats: total credits held, agents with/without credits, low-credit agents (<3)

18. Web Router (index.php)

The front controller at index.php dispatches web page requests using PHP's match() expression. URL segments are parsed from REQUEST_URI. Authentication is enforced per route: if the session is invalid, the user is redirected to /login. Role mismatches redirect to the appropriate dashboard.

Auth Routes

Path	Auth	Description
`/login`	None	Render login page (with demo login buttons)
`/register/customer`	None	Customer registration form
`/register/vendor`	None	Vendor registration form
`/register/delivery`	None	Delivery agent registration form
`/logout`	Session	Destroy session → redirect to /login

Customer Routes

Path	Auth	Description
`/customer/dashboard`	customer	Dashboard: my orders + create order shortcut
`/customer/order`	customer	Order creation form
`/customer/orders`	customer	Order history list
`/customer/track/{orderId}`	customer	Single order tracking page (polls API)
`/customer/nearby`	customer	Nearby vendor map/list

Vendor Routes

Path	Auth	Description
`/vendor/orders`	vendor	Open orders (bidding) + won orders
`/vendor/history`	vendor	Completed order history

Delivery Agent Routes

Path	Auth	Description
`/delivery/tasks`	delivery_agent	Assigned orders list
`/delivery/credits`	delivery_agent	Credit balance + purchase packages
`/delivery/task/{orderId}`	delivery_agent	Single task detail + swipe-to-complete

Admin Routes

Path	Auth	Description
`/admin`	admin	Dashboard: stats + recent orders + credit alerts
`/admin/orders`	admin	Order management panel
`/admin/users`	admin	User CRUD panel
`/admin/buildings`	admin	Building CRUD panel
`/admin/settings`	admin	Platform settings form

`/admin/credit-packages`	admin	Credit package management

Shared Routes

Path	Auth	Description
`/notifications`	any	Notification feed
`/profile`	any	User profile + edit form
`/chat/{taskId}`	any	Chat for a specific order

19. Frontend JavaScript

assets/js/app.js

- **Dark Mode** — Cookie-based toggle (td_dark). Adds/removes dark class on HTML element. Buttons: #dark-toggle (sidebar) and #dark-mode-btn (mobile nav).
- **Toast System** — showToast(title, msg, type) — injects a positioned toast overlay. Types: info (amber), success (green), error (red), warning (orange). Auto-dismisses after 4 seconds.
- **Notification Polling** — Fetches /api/v1/notifications/index.php every 15 seconds. Updates badge count. Triggered from navbar badge element.
- **Credit Balance Polling** — Fetches /api/v1/credits/balance.php every 30 seconds for delivery_agent role. Colors balance badge: green (>=5), amber (2–4), red (0–1).
- **Auth Guard** — On DOMContentLoaded: checks localStorage for JWT token. If missing and on a protected path, redirects to /login.

assets/js/chat.js

- **initChat(appUrl, token, taskId, userId)** — Sets up chat polling loop (every 3 s). Fetches messages since last seen ID. Renders bubbles (own vs other). Marks read.
- **sendMessage()** — POST to /api/v1/chat/send.php with message body. Clears input on success.

assets/js/swipe.js

- **initSwipeToComplete(containerId, thumbId, fillId, callback)** — Touch + mouse drag gesture. Thumb slides right across fill track. On full swipe (>80%), calls callback. Used on delivery task detail page for 'Confirm Delivery'.

assets/js/task-feed.js

- **Feed Diffing** — Compares new order list vs rendered DOM. Animates in new cards, removes completed ones. Used on vendor open orders page.
- **Animation Helpers** — fadeIn / slideDown for smooth list updates without full re-render.

20. Security & Error Handling

20.1 Security Measures

SQL Injection Prevention
PDO prepared statements with named placeholders throughout all models and API files. No raw string interpolation in queries.

Password Security
PASSWORD_BCRYPT via password_hash() on registration/reset. password_verify() on login. Never stored or returned in plain text.

JWT Security
HS256 signature verification via hash_equals() (timing-safe). Token expiry enforced. Secret loaded from .env (not hard-coded).

Input Sanitization
sanitizeString() = trim + strip_tags + mb_substr. validatePhone() = digits only 10–15. validateEnum() = whitelist check.

Rate Limiting
File-based rate limiter (temp files). 60/min default, 20/min login, 10/min register. Returns 429 Too Many Requests.

CORS
cors.php sets Access-Control-Allow-Origin: * with OPTIONS preflight support. Suitable for localhost dev; restrict in production.

Concurrency Safety
SELECT ... FOR UPDATE in transactions for bid selection, credit deduction, and agent assignment. PDO::ATTR_ERRMODE = EXCEPTION.

Access Control
requireAuth() enforces role arrays. Order-level ownership validated per endpoint. Chat participants verified before message access.

20.2 Standard Error Response Format
All API errors return JSON in this shape:

```
{ "success": false, "message": "Human-readable error", "errors": { "field": "validation detail" } }
```

HTTP Code	Meaning
400 Bad Request	Missing required field, invalid value, business rule violation
401 Unauthorized	Missing or invalid JWT token
403 Forbidden	Valid token but insufficient role
404 Not Found	Resource ID not found
405 Method Not Allowed	Wrong HTTP method
409 Conflict	Duplicate (phone, bid, rating)

429 Too Many Requests	Rate limit exceeded
500 Internal Server Error	PDO exception or uncaught error

21. Seed Data & Setup

21.1 Setup Steps

- **Step 1:** Start XAMPP: ensure Apache and MySQL are running
- **Step 2:** Create database: run CREATE DATABASE taskdoor; in phpMyAdmin or MySQL CLI
- **Step 3:** Import schema: run database/schema.sql
- **Step 4:** Import seeds: run database/seeds.sql
- **Step 5:** Credits migration: run database/migration_credits.sql (adds credit_packages + credit_transactions tables)
- **Step 6:** Configure .env: set DB_USER, DB_PASS, JWT_SECRET (copy .env.example if provided)
- **Step 7:** Hash passwords: visit http://localhost/aprtment/setup.php or fix-passwords.php to BCRYPT-hash seed passwords
- **Step 8:** Enable mod_rewrite: confirm Apache has mod_rewrite enabled (default in XAMPP)
- **Step 9:** Access app: http://localhost/aprtment/

21.2 Test Accounts (after password hashing)

Role	Name	Phone	Password	Details
admin	Admin	08000000000	admin123	Platform administrator
customer	Amara	08011111111	password123	Greenview Towers, unit 5B
customer	Chidi	08022222222	password123	Greenview Towers
customer	Ngozi	08033333333	password123	Sunrise Residences
vendor	Tunde Mart	08044444444	password123	Serves buildings 1 + 2
vendor	QuickShop	08055555556	password123	Serves buildings 1 + 2
vendor	Kemi Supplies	08066666666	password123	Serves buildings 2 + 3
delivery_agent	Bola	08077777777	password123	Serves buildings 1 + 2
delivery_agent	Dayo	08088888888	password123	Serves buildings 2 + 3

21.3 Test Buildings (Seeded)

ID	Name	Address	Service Radius
1	Greenview Towers	Lekki Phase 1, Lagos	2 km
2	Sunrise Residences	Lekki Phase 1, Lagos	2 km
3	The Pinnacle	Lekki Phase 1, Lagos	2 km